

RideSure

Build Guide

A step-by-step playbook to build the trust-first ride-sharing platform

Version 1.0

How to Use This Guide

This document is a practical, phase-by-phase build plan — follow it top to bottom while developing RideSure. Each phase has a checklist; tick items off as you complete them (✓) before moving to the next phase. Sections 2–4 are reference material (stack setup, folder structure, database schema, API list) you'll come back to throughout the build.

1. Tech Stack & Setup

Layer	Technology	Purpose
Frontend	React + Tailwind CSS	UI components and styling
Backend	Node.js + Express.js	REST API and business logic
Database	MongoDB	Primary data store (users, rides, bookings)
Cache	Redis	Sessions, rate limiting, hot queries
Real-time	Socket.IO	Live tracking, chat, notifications
Maps	Google Maps APIs	Geocoding, routes, live location
Payments	Razorpay	Online payments and payouts
Media storage	Cloudinary	Profile photos, ID docs, vehicle images
Push notifications	Firebase	Mobile/web push notifications
Hosting	AWS	Deployment and infrastructure

Initial setup checklist

- ☐ Create GitHub repo with frontend/ and backend/ folders
- ☐ Initialize React app (Vite) and configure Tailwind CSS
- ☐ Initialize Node.js + Express backend with a basic health-check route
- ☐ Set up MongoDB Atlas cluster and connect via Mongoose
- ☐ Set up Redis instance (local Docker for dev, managed Redis for prod)
- ☐ Create accounts: Google Cloud (Maps), Razorpay, Cloudinary, Firebase, AWS
- ☐ Add a .env file (and .env.example) for all API keys/secrets
- ☐ Set up ESLint + Prettier for consistent code style

2. Project Structure

Backend

```

backend/
  src/
    config/      # db, redis, firebase, cloudinary configs
    models/      # User, Ride, Booking, Review, Verification
    routes/      # auth, rides, bookings, users, payments, ai
    controllers/ # request handlers per route
    services/    # business logic, AI service calls, payment logic
    middleware/  # auth guard, error handler, rate limiter
    sockets/     # Socket.IO event handlers (chat, tracking)

```

```

  utils/      # helpers (OTP, JWT, validation)
  server.js

```

Frontend

```

frontend/
  src/
    components/  # shared UI components
    pages/       # Home, Search, RideDetails, Booking, Profile
    features/    # auth, rides, bookings (state + API calls)
    hooks/       # custom React hooks
    services/    # API client, socket client
    App.jsx

```

3. Database Schema (MongoDB Collections)

Collection	Key Fields
users	name, phone/email, password/oauthId, role (passenger/driver), profilePhoto, trustScore, isVerified
verifications	userId, govIdDoc, faceMatchStatus, vehicleDocs, status
rides	driverId, origin, destination, route, dateTime, seatsAvailable, price, status
bookings	rideId, passengerId, seatsBooked, status, paymentId, cancellationInfo
reviews	rideId, fromUserId, toUserId, rating, comment
payments	bookingId, amount, razorpayOrderId, status
sos_alerts	userId, rideId, location, timestamp, contactsNotified
messages	rideId, senderId, text, translatedText, timestamp

4. Core API Endpoints

Method & Path	Description
POST /api/auth/otp	Send/verify OTP for login
POST /api/auth/google	Google OAuth login
GET /api/users/me	Get current user profile
PUT /api/users/me	Update profile
POST /api/verification/id	Submit ID for verification
POST /api/verification/face	Submit face verification selfie
POST /api/rides	Driver creates a ride
GET /api/rides/search	Search rides by route/date
GET /api/rides/:id	Get ride details
POST /api/bookings	Book seats on a ride

Method & Path	Description
PUT /api/bookings/:id/cancel	Cancel a booking
POST /api/payments/order	Create payment order
POST /api/payments/verify	Verify payment after checkout
POST /api/reviews	Submit a rating/review
POST /api/sos	Trigger emergency SOS

5. Phase-by-Phase Build Plan

Phase 1 — MVP

- ☐ Auth: OTP + Google login
- ☐ User profile create/edit
- ☐ Driver: create ride listing
- ☐ Passenger: search rides by route/date
- ☐ Booking + cancellation flow
- ☐ Ride details page
- ☐ Razorpay payment integration
- ☐ Ratings & reviews after ride completion
- ☐ Push notifications for booking events
- ☐ Deploy MVP to staging on AWS

Phase 2 — Verification, Tracking, Chat & Safety

- ☐ Government ID verification flow
- ☐ Face verification (selfie match)
- ☐ Vehicle document verification for drivers
- ☐ Live trip tracking via Socket.IO + Google Maps
- ☐ In-app chat between driver and passenger
- ☐ Emergency SOS + emergency contacts
- ☐ Masked calling integration

Phase 3 — AI Trust Score & Matching

- ☐ Build AI trust score model (verification + ratings + history)
- ☐ Cancellation risk prediction
- ☐ Intelligent route matching for search results
- ☐ Smart pickup point suggestions
- ☐ AI chatbot for first-line customer support
- ☐ Fraud detection on profiles and payments

Phase 4 — Premium Safety & Women-First Options

- ☐ Women-only ride toggle and filtering
- ☐ Family live-tracking share links

- ☐ Reliability badges on profiles
- ☐ Ride insurance option at checkout
- ☐ Lost & found reporting system

Phase 5 — Full AI-Powered Mobility Platform

- ☐ Demand prediction for driver supply
- ☐ Route optimization across multiple stops
- ☐ Personalized ride recommendations
- ☐ Multilingual chat translation
- ☐ Student/community verification tiers
- ☐ Corporate carpool accounts and billing
- ☐ Carbon footprint tracking and rewards

6. Testing & Launch Checklist

- ☐ Unit tests for auth, booking, and payment logic
- ☐ End-to-end test of search → book → pay → rate flow
- ☐ Load test ride search under concurrent users
- ☐ Security review of ID/verification data storage
- ☐ Verify SOS flow with real device GPS
- ☐ Confirm compliance with local ride-hailing/KYC rules
- ☐ Set up monitoring/alerting on AWS before public launch